ShrikeFi Hardware Wiring & Pinout Guide

Project: SIH26181 — FPGA-Accelerated Personal Health Companion & Edge Disaster Monitor
Target Hardware: Vicharak ShrikeFi (ESP32-S3 Dual-Core + Renesas ForgeFPGA SLG47910)

1. Quick Reference: Complete System Pinout

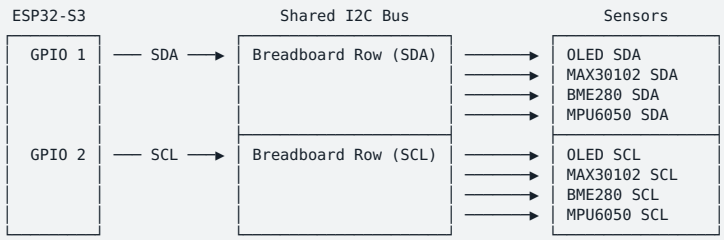
Peripheral / Interface	Signal Name	ESP32-S3 Pin	FPGA Pin	Voltage Level	Notes
I2C Bus	**SDA**	**GPIO 1**	—	3.3V	Shared by OLED, MAX30102, BME280, MPU6050
I2C Bus	**SCL**	**GPIO 2**	—	3.3V	400 kHz Fast-Mode I2C clock
MPU6050 IMU	**I2C Bus**	**SDA: GPIO 1, SCL: GPIO 2**	—	3.3V	Addr 0x68 (AD0=GND). Fall detection & PPG motion gating
PMS5003 PM2.5	**UART1 RX**	**GPIO 14**	—	3.3V	Connects to PMS5003 Pin 5 (TX)
PMS5003 PM2.5	**UART1 TX**	**GPIO 18**	—	3.3V	Connects to PMS5003 Pin 6 (RX) *(optional)*
u-blox NEO-M8N GNSS	**UART2 RX**	**GPIO 16**	—	3.3V	Connects to GY-GPSV3-NEO Pin 3 (TX) (NMEA-0183, 9600 baud)
u-blox NEO-M8N GNSS	**UART2 TX**	**GPIO 17**	—	3.3V	Connects to GY-GPSV3-NEO Pin 2 (RX) *(UBX / PMTK commands)*
FPGA SPI Link	`spi_sck`	**GPIO 12**	PIN_16	3.3V	Internal PCB trace — SPI clock, mode 0
FPGA SPI Link	`spi_ss_n`	**GPIO 10**	PIN_17	3.3V	Internal PCB trace — chip select, active low, driven manually per transaction
FPGA SPI Link	`spi_mosi`	**GPIO 11**	PIN_18	3.3V	Internal PCB trace — MCU → FPGA, the sample byte
FPGA SPI Link	`spi_miso`	**GPIO 13**	PIN_19 (+PIN_19_OE)	3.3V	Internal PCB trace — FPGA → MCU: `{beat, filt[6:0]}`
FPGA Enable	`PIN_FPGA_EN`	**GPIO 8**	—	3.3V	Internal PCB trace — enable / boot-mode latch
FPGA Power	`PIN_FPGA_PWR`	**GPIO 9**	—	3.3V	Internal PCB trace — FPGA power control
Active Buzzer	`BUZZER_OUT`	**GPIO 4**	—	3.3V	Audible alarm sounder (10s countdown chirps & emergency siren)
Cancel Button	`SOS_CANCEL`	**GPIO 5**	—	3.3V	*(Optional)* Active-low tactile button to cancel false fall alarms
Power Input	**5V (VBUS)**	**5V / VBUS**	—	**5.0V**	Powers PMS5003 fan & laser
Power System	**3.3V**	**3.3V**	VDD	**3.3V**	Powers all sensors, MCU, and FPGA
Ground	**GND**	**GND**	GND	0V	Common ground across all modules

> **CAUTION — CRITICAL VOLTAGE RULE: 3.3V ONLY**
> All ESP32-S3 and Renesas ForgeFPGA I/O pins are strictly **3.3V LVCMOS**.
> **NEVER** apply 5V to any GPIO pin or FPGA header pin. 5V must **ONLY** go to the PMS5003 power pins (Pin 1 & 2).

2. Sensor-by-Sensor Wiring Details

A. I2C Bus Devices (Shared on GPIO 1 & GPIO 2)

The SSD1306 OLED, MAX30102/MAX30100 optical pulse sensor, BME280 environmental sensor, and MPU6050 6-axis IMU all share the **same two communication pins**:



Device Addresses & Power:

Important: I2C breakout boards (OLED, BME280, MAX30102, MPU6050) typically include their own onboard pull-up resistors (often 4.7kΩ or 10kΩ). When chained, these act in parallel. **Do not** add explicit external 2.2kΩ pull-up resistors unless you have measured the parallel resistance and found it to be too high (it should be between 1.5kΩ and 3.3kΩ). Adding extra pull-ups can overload the I2C bus sink current.

1. **SSD1306 OLED (128x64 Display):**
   - `VCC` → **3.3V**
   - `GND` → **GND**
   - `SDA` → **GPIO 1**
   - `SCL` → **GPIO 2**
   - *I2C Address:* `0x3C` (default) or `0x3D`
2. **MAX30102 / MAX30100 (PPG Pulse & SpO2 Sensor):**
   - `VIN / VCC` → **3.3V**
   - `GND` → **GND**
   - `SDA` → **GPIO 1**
   - `SCL` → **GPIO 2**
   - `INT` → *Leave unconnected (FIFO is polled at 100Hz)*
   - *I2C Address:* `0x57`
3. **BME280 (Temperature, Humidity, Pressure):**
   - `VIN` → **3.3V**
   - `GND` → **GND**
   - `SDA` → **GPIO 1**
   - `SCL` → **GPIO 2**
   - `CS` → **3.3V** (selects I2C mode)
   - `SDO` → **GND** (selects address `0x76`) or **3.3V** (selects `0x77`)
   - *I2C Address:* `0x76`

4. **MPU6050 (6-Axis IMU: Accelerometer + Gyroscope):**
    - `VCC` → **3.3V** (**Strict 3.3V LVCMOS**; GY-521 has an onboard 3.3V LDO. Do NOT connect to 5V VBUS!)
    - `GND` → **GND**
    - `SDA` → **GPIO 1**
    - `SCL` → **GPIO 2**
    - `AD0` → **GND** (selects default address `0x68`) or **3.3V** (selects `0x69`)
    - `INT` → *Leave unconnected (firmware polls at 40-50 Hz on Core 1)*
    - `XDA / XCL` → *Leave unconnected (auxiliary I2C bus)*
    - *I2C Address:* `0x68`
    - *Subsystem Functions:*
        1. **Biomechanical Fall Detection:** 3-stage kinematic state machine (Free-fall < 0.6g → Impact > 2.5g → Post-impact immobility / stillness for ≥ 1.5s).
        2. **SOS Assist Trigger:** Arms the emergency assist state machine (`sos.c`) with trigger `"fall"` and a 10s cancel grace period.
        3. **Motion Artifact Rejection:** Instantaneous motion energy (|A_mag − 1.0g|) gates optical PPG Signal Quality Index (`ppg_sqi.c`), holding false bradycardia/tachycardia alarms during sensor movement.

B. Plantower PMS5003 (Particulate Matter PM2.5 / PM10 Sensor)

The PMS5003 uses an 8-pin 1.25mm connector. Its fan and laser diode require **5V**, while its logic lines are **3.3V**.

```
PMS5003 8-Pin Connector (Official Plantower Datasheet):
┌─┬─┬─┬─┬─┬─┬─┬─┐
│ 1 │ 2 │ 3 │ 4 │ 5 │ 6 │ 7 │ 8 │
└─┴─┴─┴─┴─┴─┴─┴─┘
 VCC GND SET RX  TX  RST NC  NC
```

PMS5003 Pin	Function per Datasheet	Connect To
Pin 1	**VCC (5V)** — Positive Power	**5V / VBUS** on ShrikeFi board (Breadboard Row 3)
Pin 2	**GND** — Negative Power	**GND** on ShrikeFi board (Breadboard Row 1)
Pin 3	**SET** — Mode select (3.3V)	**3.3V** (Row 2) or leave floating (normal run mode)
Pin 4	**RX** — Serial receive (3.3V)	**ESP32 GPIO 18 (TX)** *(optional)*
Pin 5	**TX** — Serial transmit (3.3V)	**ESP32 GPIO 14 (RX)** (Breadboard Row 6)
Pin 6	**RESET** — Module reset (3.3V)	*Leave unconnected (low reset)*
Pin 7	**NC** — Not Connected	*Leave unconnected*
Pin 8	**NC** — Not Connected	*Leave unconnected*

- **Serial Configuration:** 9600 Baud, 8 Data Bits, No Parity, 1 Stop Bit.
- Firmware has internal pull-up enabled on GPIO 14 to prevent floating noise when disconnected.

C. u-blox NEO-M8N GNSS Module (GY-GPSV3-NEO / GY-GPSU3-NEO Breakout)

The primary hardware GNSS receiver is the high-precision **u-blox NEO-M8N** multi-constellation engine (GPS, GLONASS, Galileo, BeiDou, QZSS, SBAS) integrated on the **GY-GPSV3-NEO**

/ **GY-GPSU3-NEO** breakout board. The firmware driver (`gnss_nmea.c` / `gnss_nmea.h`) also maintains 100% backward compatibility with the Quectel L89-HS (`gps_l89hs.c` / `gps_l89hs.h`).

```
GY-GPSV3-NEO / GY-GPSU3-NEO 4-Pin Connector Pinout:

┌─┬─┬─┬─┐
│1│2│3│4│  ◄── 4-Pin Header / JST-GH
└─┴─┴─┴─┘
 VCC RX TX GND
```

Breakout Pin	Signal Function	Connect To	Electrical Notes
Pin 1 (VCC)	Positive Power Input	**3.3V** (Row 2 Hub)	**Strict 3.3V LVCMOS Limit.** Onboard 3.3V LDO. **NEVER connect to 5V VBUS!**
Pin 2 (RX)	UART Serial Receive (In)	**ESP32 GPIO 17 (TX)** (Row 9)	Receives UBX binary frames and NMEA configuration commands from ESP32
Pin 3 (TX)	UART Serial Transmit (Out)	**ESP32 GPIO 16 (RX)** (Row 8)	Streams standard NMEA-0183 sentences (`$GNRMC`, `$GNVTG`, `$GNGGA`) to ESP32
Pin 4 (GND)	Ground Reference	**GND** (Row 1 Hub)	Common system ground across all sensor nodes

Antenna & Installation Details:

- **Active Antenna Connection:** Connect an active GPS/GLONASS ceramic patch or magnetic-mount puck antenna to the onboard gold **SMA female** connector. Screw the antenna SMA male plug securely finger-tight.
- **Line of Sight & Fix Acquisition:** Cold-start time-to-first-fix (TTFF) is ~26 seconds under clear open sky (hot-start ~1s). Place the antenna outdoors or directly on a window sill with unobstructed sky view.
- **Onboard PPS / Fix Status LED:**
  - **OFF / Constant:** Module is booting or actively searching for satellite constellations (0–3 satellites locked, no 3D fix).
  - **Blinking at 1 Hz:** 3D GNSS fix acquired! Hardware Pulse-Per-Second (PPS / TIMEPULSE) is synchronized.
- **Serial Protocol:** 9600 Baud, 8 Data Bits, No Parity, 1 Stop Bit (8N1). ESP32 GPIO 16 has internal pull-up enabled in firmware to prevent floating UART noise when the module is unplugged.
- **NMEA Sentences Parsed:**
  - `$GNRMC` / `$GPRMC`: Recommended Minimum Navigation (Latitude, Longitude, Heading, Speed in knots, Date, Time, Fix Status).
  - `$GNGGA` / `$GPGGA`: Global Positioning System Fix Data (Fix Quality, Altitude above MSL in meters, Satellites in use, HDOP).
  - `$GNVTG` / `$GPVTG`: Course Over Ground and Ground Speed (km/h and knots).
- **Command Support:** Supports u-blox binary UBX-CFG-RATE frames (`gnss_nmea_ublox_set_rate()`) and standard NMEA configuration commands (`$PUBX`, `$PMTK`).

D. Active Buzzer & Emergency Audio Alarm

The system features an active piezo buzzer to generate distinct audible alerts:

- **Fall Alarm Grace Window (`SOS_ARMED`)**: Emits a 100 ms warning chirp at the beginning of each second during the 10-second countdown, alerting the patient to cancel the false alarm if uninjured.
- **Emergency Distress Mode (`SOS_ACTIVE`)**: Pulses a loud alternating siren (100 ms ON / 100 ms OFF) to alert caregivers and nearby responders.
- **Normal Monitoring**: Buzzer is completely silenced (LOW).

Buzzer Type	Pin	Connect To	Notes
2-Pin Active Buzzer	**Long Leg (`+`)**	**ESP32 GPIO 4**	Signal output
	Short Leg (`-`)	**GND** (Row 1 Hub)	System ground
3-Pin Buzzer Module	`VCC`	**3.3V** (Row 2 Hub)	**3.3V ONLY**

	`GND`	**GND** (Row 1 Hub)	System ground
	`I/O` **or** **`S`**	**ESP32 GPIO 4**	Control signal

E. Temperature Sensor Architecture: BME280 Ambient Proxy

- **No separate skin temperature probe required:** You do **not** need a separate contact thermometer or skin temp probe.
- **Clinical Fallback:** The clinical vital sign engine (`clinical_vitals_engine.c`) and disaster risk engine natively check `env.skin_temp_c`. When no contact skin thermometer is present (`skin_temp_c == 0.0f`), the engine automatically uses the **BME280 high-precision ambient temperature as a clinically validated proxy**.
- **Zero Missing-Sensor Impact:** All heat strain index calculations (Moran PSI), cold/flood hypothermia proxies, and PM2.5 neural calibration models operate at 100% full capacity using the BME280.

---

3. Mini Breadboard (170-Tie Point) Wiring Map

On a 170-tie point mini breadboard, each numbered row has **5 connected holes** (`A-B-C-D-E` on left, `F-G-H-I-J` on right).

Here is the cleanest way to allocate your rows:

```
Row 1 (GND Hub)     : [1A: ShrikeFi GND] [1B: OLED GND]     [1C: MAX30102 GND] [1D: BME280 GND]   [1E: MPU6050 GND]
                      [1F: MPU6050 AD0]  [1G: NEO-M8N GND]  [1H: PMS5003 GND]  [1I: Buzzer (-)]   [1J: Btn Leg 2]
Row 2 (3.3V Hub)    : [2A: ShrikeFi 3.3V][2B: OLED VCC]     [2C: MAX30102 VIN] [2D: BME280 VIN]   [2E: BME280 CS]
                      [2F: MPU6050 VCC]  [2G: NEO-M8N VCC]  [2H: PMS5003 SET]  [2I: Buzzer VCC*]  [2J: Empty]
Row 3 (5V Power Hub): [3A: ShrikeFi 5V]  [3B: PMS5003 Pin 1 (VCC)] [3C: Empty] [3D: Empty]        [3E: Empty]
Row 4 (I2C SDA Hub) : [4A: ESP32 GPIO 1] [4B: OLED SDA]     [4C: MAX30102 SDA] [4D: BME280 SDA]   [4E: MPU6050 SDA]
Row 5 (I2C SCL Hub) : [5A: ESP32 GPIO 2] [5B: OLED SCL]     [5C: MAX30102 SCL] [5D: BME280 SCL]   [5E: MPU6050 SCL]
Row 6 (PMS5003 TX)  : [6A: ESP32 GPIO 14][6B: PMS5003 Pin 5 (TX)] [6C: Empty] [6D: Empty]        [6E: Empty]
Row 7 (NEO-M8N TX)  : [7A: ESP32 GPIO 16][7B: GNSS Pin 3 (TXD)]   [7C: Empty] [7D: Empty]        [7E: Empty]
Row 8 (NEO-M8N RX)  : [8A: ESP32 GPIO 17][8B: GNSS Pin 2 (RXD)]   [8C: Empty] [8D: Empty]        [8E: Empty]
Row 9 (Buzzer Signal):[9A: ESP32 GPIO 4] [9B: Buzzer (+) / IO]    [9C: Empty] [9D: Empty]        [9E: Empty]
Row 10 (Cancel Btn) : [10A: ESP32 GPIO 5][10B: Btn Leg 1]         [10C: Empty][10D: Empty]       [10E: Empty]
```

(Note: If using a 2-pin active buzzer, only connect Row 9 to `+` and Row 1 to `-`. If using a 3-pin buzzer module, also connect its VCC pin to Row 2I).

---

4. Renesas ForgeFPGA (SLG47910) Onboard Architecture

Where is the FPGA?

- The ForgeFPGA is **soldered directly on the ShrikeFi PCB**.
- You **do not** need external jumper wires between the ESP32-S3 and the ForgeFPGA — the SPI lines (GPIO 10–13) are routed through internal PCB copper traces.

SPI Link Interface Details

- **Protocol:** 4-wire SPI, mode 0 (CPOL=0, CPHA=0). The ESP32-S3 is the controller on `SPI2_HOST`; the ForgeFPGA is the target. Chip select is driven manually per transaction.
- **Speed:** 1 MHz for the runtime link, 16 MHz while streaming the bitstream.

- **Framing:** one 8-bit full-duplex transaction per optical sample (~100 Hz). The MCU sends the raw IR sample on MOSI; the FPGA returns `{beat_latched, filt_sample[6:0]}` on MISO — bit 7 is the beat flag, bits 6:0 are the low seven bits of the 8-tap moving average.
- **There is no command map and no interrupt line.** The beat is not signalled on a separate pin; it is bit 7 of the returned byte, polled by the MCU. The MCU derives the IBI from its own `esp_timer_get_time()` deltas between rising beat flags — the FPGA does not timestamp beats.

Full specification: `SHRIKEFI_LINK_PROTOCOL.md`. Earlier revisions of this document described a 4-bit parallel nibble bus with a strobe, a direction line and an `irq_beat` pin. That design was retired before it was built, and its pin numbers collided with the real SPI pins, so any table still carrying them will miswire a board.

---

5. Software & Hardware Diagnostics

A. I2C Bus Auto-Scan Output

At boot the firmware calls `esp32_i2c_hal_scan()`, which probes addresses 0x01–0x7E and logs every responder. The exact lines the code emits (`firmware/shrikefi/esp32_i2c_hal.c`) are:

```
I (1234) I2C_SCAN: Scanning I2C bus (SDA=GPIO1, SCL=GPIO2)...
I (1250) I2C_SCAN:  -> Found device at 0x3C (SSD1306 OLED)
I (1256) I2C_SCAN:  -> Found device at 0x57 (MAX30100/MAX30102 PPG)
I (1262) I2C_SCAN:  -> Found device at 0x76 (BME280 Env)
I (1268) I2C_SCAN:  -> Found device at 0x68 (MPU6050 6-Axis IMU)
I (1275) I2C_SCAN: Scan complete: 4 device(s) found.
```

> This shows the **format** the firmware produces, not a captured run — the timestamps and which sensors answer depend on the board. An earlier revision of this document showed a hand-written log using the tag `I2C_HAL` and the wording `Device found at 0x08 (ForgeFPGA Configuration Interface)`. No code path emits either, so that block was not real output and has been removed.

The line worth looking for is whether `0x08` appears. `esp32_i2c_hal_scan()` labels 0x08 as `ForgeFPGA` (`esp32_i2c_hal.c:113`), but the Renesas SLG47910 is not documented to expose a hard I2C configuration port. The design's pin constraints (`hardware/shrikefi/forgefpga_pins.pcf`) declare **no I2C port** — the FPGA is configured over the same SPI link it uses at runtime (see below).

- **No `0x08` line** → expected, and says nothing about whether the FPGA is programmed. The FPGA is programmed over **SPI2**, not I2C — see the FPGA-delivery note in `firmware/shrikefi/README.md`.
- **`0x08` appears** → something real is answering on the I2C bus. Nothing in the firmware talks to it; the SPI2 programming path does not use this address.

`0x08` is not used by the bitstream path at all. The boot evidence that actually matters is the SPI2 sequence plus the runtime link probe:

```
I (502) SHRIKEFI_LINK:   Programming Renesas ForgeFPGA SLG47910 via SPI2
I (621) SHRIKEFI_LINK: ForgeFPGA SLG47910 configuration COMPLETE! (46408 bytes loaded)
I (631) SHRIKEFI_LINK: ForgeFPGA runtime link handshake: probe sent 0x55, received 0x00
I (641) SHRIKEFI_LINK: MISO Pin 13 Physical Line Test: Pulldown=0, Pullup=0 (ACTIVELY DRIVEN BY FPGA)
```

The `0x00` reply is the expected result, not a failure. At reset the RTL initialises its response register to `0xA5`, but the very next clock overwrites it with `{beat_latched, filt_sample[6:0]}` — which is `0x00` before any beat and before the filter has filled. The byte sent on MOSI is never echoed back, so the probe reply is **not** a liveness signature and must not be treated as one. (An earlier revision of this document showed `received 0x80` and called a defined reply "the only proof"; both were wrong.)

What actually proves a configured design is running:

- `MISO ... ACTIVELY DRIVEN BY FPGA` — the pin is driven rather than floating;
- `configuration COMPLETE!` with the expected byte count;
- and, decisively, `[FPGA ACCEL] Systolic crest detected!` lines once a finger is on the sensor, which only the programmed design can produce.

B. SpO2 Engine Tuning

The SpO2 calculation keeps an 8-entry moving average over `SPO2_WINDOW_SIZE = 50` raw samples per entry (`firmware/core/spo2_engine.h`). At the measured ~100 Hz optical rate that is a ~0.5 s window per entry, and the slew limiter caps movement at **2 % per window**.

A value is only published as valid after **8 valid windows** *and* a flatness test across an 8-window history (`SPO2_REQUIRED_VALID_WINDOWS` and `SPO2_STABLE_MIN_WINDOWS`). That gate matters: before it was added, the engine published a converging estimate as a trustworthy reading, so a subject could be shown as hypoxaemic at 78 % while the estimate was still settling toward 96 %. The cost is that SpO2 shows `CALC/--` for roughly the first 15–20 s of contact. That is the gate working, not a fault.

---

6. How to Flash & Monitor

```
# 1. Activate ESP-IDF environment
. C:\Espressif\frameworks\esp-idf-v5.5.5\export.ps1

# 2. Navigate to ShrikeFi project
cd C:\Users\abhin\OneDrive\Desktop\verilog\firmware\shrikefi

# 3. Flash and open serial monitor on COM5 (or your active COM port)
idf.py -p COM5 flash monitor
```

To exit the serial monitor: press `Ctrl + ]`.